

Homework 5 Report

Group 09

Members: 徐柏安、鄭家齊、許維也、顏名柔、鄒政昇、黃柏翔

1. Introduction

This report describes our ontology-based semantic grounding design for Homework 5. The goal of this homework is to construct a small but explicit ontology for grounding task objects in our final project. The ontology connects object types, task roles, manipulation affordances, and inferred graspability or pressability.

The ontology covers all three baseline manipulation tasks: cup stacking, cutlery arrangement, and toy block collection. We add one advanced task: pump bottle pressing. Class memberships for **cap:GraspableObject** and **g09:PressableObject** are derived entirely through a Python-based materialisation workflow using RDFLib; no instance is manually typed as graspable or pressable in the asserted graph

2. Repository

2.1 Repository Structure

Our repository at https://github.com/cs12-richard/AI_capstone_hw5 is organized as follows:

```
semantic-affordance-grounding/
|-- README.md
|-- report.pdf
|-- ontology/
|   |-- group-ontology.ttl
|   |-- inferred-results.ttl
|   `-- imports/
|       `-- course-affordance.ttl
|-- queries/
|   |-- graspable_objects.rq
|   |-- pressable_objects.rq
|   `-- pump_bottle_query.rq
|-- results/
|   |-- graspable_objects_output.txt
|   `-- screenshots/
|-- src/
|   `-- ...
`-- LICENSE
```

2.2 Repository Content

Path	Description
README.md	Project overview: namespace policy, query instructions, expected output

report.pdf	This report
ontology/group-ontology.ttl	Group-authored ontology: namespace, instances, affordance individuals, class extensions
ontology/inferred-results.ttl	Inferred RDF graph exported by <code>reason_and_export.py</code> after materialisation
ontology/imports/course-affordance.ttl	Provided course ontology (imported dependency)
queries/graspable_objects.rq	SPARQL query for inferred <code>cap:GraspableObject</code> individuals
queries/pressable_objects.rq	SPARQL query for inferred <code>g09:PressableObject</code> individuals (advanced)
queries/pump_bottle_query.rq	SPARQL query for inferred pump bottle. (advanced)
results/graspable_objects_output.txt	Saved output of graspable query
results/pressable_objects_output.txt	Saved output of pressable query
src/reason_and_export.py	Python materialisation script (RDFLib + custom forward-chaining rule)

3. Ontology Design

3.1 Namespace Policy

We use two namespace prefixes:

□ **cap:**

```
@prefix cap: <https://hcis.io/ontology/aicapstone/2026/> .
```

This namespace is reserved for shared course vocabulary.

□ **g09:**

```
@prefix g09: <https://hcis.io/ontology/aicapstone/2026/group09/> .
```

Other group-specific terms, including classes, properties, affordances, tasks, and individuals are under **g09:** namespace. For example, `g09:pumpBottle01`, `g09:PressingAffordance`, and `g09:PressableObject`.

3.2 Modeling Layers

We divide the ontology into four semantic layers to avoid conflating object, task functions, affordance, and individuals. Following are the modeling layers we use:

Layer	Description	Examples
Object Type	Shared OWL class for an object category	cap:Cup, cap:Knife, cap:ToyBlock, g09:PumpBottle
Task Role	Role an object plays in a specific task	cap:TargetObject, cap:ReferenceObject, cap:ContainerTarget, cap:CollectableObject
Affordance	Manipulation capability of an object	cap:GraspingAffordance, cap:SupportAffordance, cap:ContainmentAffordance, g09:PressingAffordance
Instance	Concrete observed or simulated object	g09:blueCup01, g09:knife01, g09:block01, g09:pumpBottle01

3.3 Class Hierarchy

All task-object classes inherit from **cap:PhysicalObject**. In addition to the course classes, we add one new object class:

```
g09:PumpBottle rdfs:subClassOf cap:PhysicalObject
```

The owl:equivalentClass axioms for cap:GraspableObject and g09:PressableObject are specified in Section 4.

3.4 Modeled Objects and Their Affordances

Instance (g09:)	Type	Task Role	Affordances
blueCup01	cap:Cup	TargetObject	GraspingAffordance, StackabilityAffordance
pinkCup01	cap:Cup	TargetObject	GraspingAffordance, StackabilityAffordance
knife01	cap:Knife	TargetObject	GraspingAffordance
fork01	cap:Fork	TargetObject	GraspingAffordance
plate01	cap:Plate	ReferenceObject	SupportAffordance
block01	cap:ToyBlock	CollectableObjec	GraspingAffordance

		t	
basket01	cap:Basket	ContainerTarget	ContainmentAffordance
pumpBottle01	g09:PumpBottle	TargetObject	g09:PressingAffordance

3.5 Design Choices

□ **Cup (g09:blueCup01, g09:pinkCup01)**

These two instances are the direct manipulation targets in the cup-stacking and carry cap:GraspingAffordance and cap:StackabilityAffordance. In addition, the stacking affordance reflects that cups can be nested after being grasped and repositioned. Therefore, are inferred as cap:GraspableObject .

□ **Knife and Fork (g09:knife01, g09:fork01)**

These two instances are direct placement targets in the cutlery arrangement task and carry cap:GraspingAffordance. Therefore, they are inferred as cap:GraspableObject.

□ **Block (g09:block01)**

The toy block is the collectable grasp target in the block collection task and carries cap:GraspingAffordance. Therefore, it is inferred as cap:GraspableObject.

□ **Plate and Basket (g09:plate01, basket01)**

The plate is a placement reference surface (cap:ReferenceObject, cap:SupportAffordance). And the basket is a container target (cap:ContainerTarget, cap:ContainmentAffordance). Neither of them carries cap:GraspingAffordance; therefore, they aren't inferred as cap:GraspableObject. This reflects the task semantics: the robot only interacts with them but does not grasp themselves in the baseline task definition.

□ **Pump bottle (g09:pumpBottle01)**

This is our advanced-task object. It belongs to class g09:PumpBottle and carries g09:PressingAffordance. Therefore, the script derives g09:PressableObject for it. Instead of picking the bottle up, the robot presses the pump head while the bottle remains upright, so no GraspingAffordance is asserted.

3.6 Reused and Newly Introduced Terms

□ Reused course terms:

Term	Kind	Description
cap:PhysicalObject	owl:Class	Root superclass for all task objects
cap:Cup, cap:Knife, cap:Fork, cap:Plate, cap:ToyBlock, cap:Basket	owl:Class	Asserted types for all baseline instances
cap:GraspableObject	owl:Class	Inferred class for graspable individuals
cap:GraspingAffordance, cap:StackabilityAffordance, cap:SupportAffordance, cap:ContainmentAffordance	owl:Class	Affordance types assigned to baseline instances
cap:TargetObject, cap:ReferenceObject, cap:ContainerTarget, cap:CollectableObject	Task Role	Task roles assigned via cap:hasTaskRole
cap:hasAffordance, cap:hasTaskRole, cap:hasObjectLabel, cap:hasPoseFrame, cap:hasColor	Properties	Used to describe all instances

□ Newly introduced terms:

Term	Kind	Description
g09:PumpBottle	owl:Class	Asserted type for the advanced-task object
g09:PressingAffordance	owl:Class	Affordance type assigned to the pump bottle instance
g09:PressableObject	owl:Class (Inferred)	Equivalent to $\text{PhysicalObject} \cap \exists \text{hasAffordance.PressingAffordance}$
g09:usedInTask	owl:ObjectProperty	Relates an object individual to its ManipulationTask instance
g09:hasGripWidthMM	owl:DatatypeProperty	Approximate gripper width in mm needed to grasp the object
g09:pumpBottlePressTask	Individual (Task)	Task instance for pump bottle pressing

4. Key Axioms and Restrictions

The central axiom driving graspability inference is the `owl:equivalentClass` definition of `cap:GraspableObject`, serialized in the group ontology as:

```
cap:GraspableObject
  a owl:Class ;
  rdfs:subClassOf cap:PhysicalObject ;
  owl:equivalentClass [
    a owl:Class ;
    owl:intersectionOf (
      cap:PhysicalObject
      [ a owl:Restriction ;
        owl:onProperty cap:hasAffordance ;
        owl:someValuesFrom cap:GraspingAffordance ]
    )
  ] .
```

The same pattern is applied to `g09:PressableObject` using `g09:PressingAffordance`.

```
cap:GraspableObject ≡ cap:PhysicalObject ⊓
  ∃ cap:hasAffordance . cap:GraspingAffordance
```

```
g09:PressableObject ≡ cap:PhysicalObject ⊓
  ∃ cap:hasAffordance . g09:PressingAffordance
```

These axioms are not executed directly by RDFLib's default SPARQL engine; the materialization script in `src/reason_and_export.py` implements the inference rule explicitly. Individual object-type subclass restrictions from the course ontology (e.g., `cap:Cup rdfs:subClassOf [owl:someValuesFrom cap:GraspingAffordance]`) are fulfilled by attaching named affordance individuals (e.g., `g09:blueCup01GraspingAffordance`) to each instance via `cap:hasAffordance`.

5. Reasoning Workflow

5.1 Selected Option: Python RDFLib with Custom Materialisation

We use Python RDFLib and implement a custom forward-chaining materialization in `src/reason_and_export.py` that explicitly derives `cap:GraspableObject` and `g09:PressableObject` memberships. This is because RDFLib does not perform full OWL 2 DL reasoning by default

5.2 Inference Algorithm

The core function `materialize_by_affordance(graph, affordance_class, inferred_class)` proceeds as follows:

- Iterate over all (subject, cap:hasAffordance, affordance_individual) triples in the merged graph.
- Retrieve all rdf:type values of the affordance individual.
- Check whether any affordance type is a subclass of the target affordance_class (via custom is_subclass_of traversal over rdfs:subClassOf chains).
- If the affordance condition is met, check whether any rdf:type of the subject is a subclass of cap:PhysicalObject.
- If both conditions hold, add (subject, rdf:type, inferred_class) to the inferred graph.

This materialization faithfully corresponds to the OWL DL expression above.

5.3 Generating inferred-results.ttl

After materialization, the script serializes only the inferred triples to ontology/inferred-results.ttl. The current ontology produces inferred cap:GraspableObject individuals and one g09:PressableObject individual.

5.4 How to Reproduce

```
# From the repository root:
pip install rdflib
python src/reason_and_export.py
```

The script loads both course-affordance.ttl and group-ontology.ttl, runs materialisation, writes ontology/inferred-results.ttl, and saves both query result files under results/.

6. SPARQL Queries

6.1 graspable_objects.rq

```
PREFIX cap: <https://hcis.io/ontology/aicapstone/2026/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?obj ?label ?role
WHERE {
  ?obj a cap:GraspableObject .
  OPTIONAL { ?obj rdfs:label ?label . }
  OPTIONAL { ?obj cap:hasTaskRole ?role . }
}
ORDER BY ?obj
```

6.2 pressable_objects.rq (Advanced Extension)

```
PREFIX cap: <https://hcis.io/ontology/aicapstone/2026/>
PREFIX g09:
<https://hcis.io/ontology/aicapstone/2026/group09/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?obj ?label ?role
WHERE {
  ?obj a g09:PressableObject .
  OPTIONAL { ?obj rdfs:label ?label . }
  OPTIONAL { ?obj cap:hasTaskRole ?role . }
}
ORDER BY ?obj
```

Both queries must be executed over the merged graph (base + inferred). Running them against the raw asserted graph only would return no results because neither class is directly asserted for any individual.

7. Query Results

7.1 Graspable Objects

Individual	Label	Task Role
g09:block01	toy block 01	cap:CollectableObject
g09:blueCup01	blue cup 01	cap:TargetObject
g09:fork01	fork 01	cap:TargetObject
g09:knife01	knife 01	cap:TargetObject
g09:pinkCup01	pink cup 01	cap:TargetObject

g09:plate01 and g09:basket01 are absent because they carry no GraspingAffordance.

7.2 Pressable Objects (Advanced)

Individual	Label	Task Role
g09:pumpBottle01	pump bottle 01	cap:TargetObject

8. Limitations

- Reasoning uses a Python/RDFLib forward-chaining materialiser rather than a full OWL-DL reasoner (e.g., HermiT). The implemented logic is faithful to the owl:equivalentClass definitions but does not cover all OWL 2 DL entailments.
- SHACL validation shapes and Protégé screenshots are not included in this submission.
- If the course workflow requires output from a specific OWL 2 reasoner, the same ontology files and queries can be reloaded into Protégé or Apache Jena Fuseki to regenerate inferred-results.ttl.

9. Discussion

Regarding the design process of this assignment and the application of the Semantic Web in the field of robotics, our team proposes the following points for discussion:

1. Extensibility of Advanced-Level Modeling

In this assignment, we utilized `g09:PressingAffordance` and `g09:PressableObject` as the core of our advanced tasks. This design pattern demonstrates the powerful extensibility of ontologies: when a robot needs to learn a new task, we do not need to refactor the entire underlying architecture; we simply introduce new Affordance classes and equivalent class reasoning rules. This Affordance-based modeling approach enables the robot to better generalize to unseen objects. For example, if a "push-button switch" or a "keyboard key" is introduced in the future, assigning it a `PressingAffordance` will allow the robot to automatically classify it as a `PressableObject`.

2. Integration of Heterogeneous Knowledge: T-Box, A-Box, and Robot Control

During the implementation process, we experienced the benefits of decoupling the T-Box from the A-Box within the knowledge graph.

- The shared course ontology (`cap:`) serves as the Upper Ontology, standardizing the standard task workflow.
- The group ontology (`g09:`) acts as the Domain Ontology, defining the specific physical scenario.

In practical applications, this architecture can function as the semantic brain for a robot task planner. Through SPARQL queries, the robot can first make decisions at the semantic level regarding what to grasp and what to press, and then pass the numerical results of these queries (such as `g09:hasGripWidthMM`) to the low-level motion planning algorithms. This effectively achieves the decoupling of high-level reasoning and low-level control.